Factory Design Pattern

↓

Factory Method is a creational design pattern
that defines an interface for creating objects,
but lets subclasses decide which object to create

Simple:
Object creation hide kar deta hai
Client ko exact class ka pata nahi hota

Core Idea--> Object kaun sa banega → ye factory decide karegi, client

Problem It Solves:

Without Factory Method

```
if (type == "EMAIL") {
    new EmailNotification();
} else if (type == "SMS") {
    new SMSNotification();
}
```

Problems:
Tight coupling
Har new type pe code change
Maintain karna mushkil
Open/Closed principle break

Jaise-jaise features badhte hain, code messy ho jata hai

Solution (Factory Method)

Instead of:
client decides object
We do:
factory/creator decides object

Structure (Important for Interviews)

1. Product
Common interface

```
interface Notification {
    void send();
}
```

2. Concrete Products

Actual implementations

```
class EmailNotification implements Notification {}
class SMSNotification implements Notification {}
```

3. Creator (Factory Class)

Defines factory method

```
abstract class NotificationCreator {
    abstract Notification createNotification();
}
```

4. Concrete Creators

Actual object decide karte hain

```
class EmailCreator extends NotificationCreator {
    Notification createNotification() {
        return new EmailNotification();
    }
}
```

Flow (Step-by-Step)
Client → Creator choose karta hai
Creator → createNotification() call karta hai
Subclass → actual object create karti hai
Creator → object use karta hai

Client ko pata hi nahi kaunsa object bana

Full Clean Code (Interview Level)

```
interface Notification {
    void send();
}

class EmailNotification implements Notification {
    public void send() {
        System.out.println("Email sent");
    }
}

abstract class NotificationCreator {
    abstract Notification createNotification();

    public void notifyUser() {
        Notification n = createNotification();
        n.send();
    }
}

class EmailCreator extends NotificationCreator {
    Notification createNotification() {
        return new EmailNotification();
    }
}
```

When to Use (Very Important)

Use Factory Method when:

1. Runtime pe object decide hota hai
payment type
notification type
2. Multiple similar objects hain
3. Code loosely coupled chahiye
4. Object creation complex hai

Advantages
1. Loose Coupling
Client ko class ka pata nahi

2. Easy Extensibility
New type add karo → old code touch nahi

3. Clean Code
No if-else jungle
Follows Open/Closed Principle

1. Disadvantages
1. More Classes
Har product ke liye creator

2. Complexity
Small apps me overkill

Real Use Cases (Important)

Payment gateways (Stripe, PayPal)
Notification systems (Email, SMS)
File parsers (JSON, XML)
Database connectors
Logging frameworks

More_notes : https://algomaster.io/learn/lld/factory-method